1-3

一、解题过程

（1）产品毛利

产品 Ⅰ：单件毛利 = 1.25−0.25=1.00 元 / 件

产品 Ⅱ：单件毛利 = 2.00−0.35=1.65 元 / 件

产品 Ⅲ：单件毛利 = 2.80−0.50=2.30 元 / 件

（2）设备费用分摊

设备费用是满负荷总费用，需按实际使用工时占比分摊：

设备 A1：满负荷 300 元，总工时 6000，单位工时费用 = 300/6000=0.05 元 / 小时

设备 A2：满负荷 321 元，总工时 10000，单工时费用 = 321/10000=0.0321 元/小时

设备 B1： 0.0625 元/小时

设备 B2：0.111857 元 / 小时

设备 B3： 0.05 元/小时

（3）决策变量定义

x111 表示产品 1 经过 A1-B1 工序的数量，x212 表示产品 2 经过 A2-B2，其余同理。

（4）目标函数

总利润（=产品总毛利-设备总费用） 最大化

1. 产品总毛利

产品 Ⅰ 总产量：X1=x111+x112+x113+x121+x122+x123 → 毛利：1.00×X1

其余同理，毛利相加得到产品总毛利

2. 设备总费用

A1 总工时：5(x111+x112+x113)+10x211 → 费用：0.05×[5(x111+x112+x113)+10x211]

其余同理，费用相加得到设备总费用

3. 总利润为产品总毛利减去设备总费用，取最大值即可

（5）约束条件

1. 设备工作时间约束

A1 台时：5(x111+x112+x113)+10x211≤6000

其余同理（共 5 台机器）

2. 非负约束

所有决策变量≥0

（6）求解模型

Python 的 pulp 库（求解线性规划问题）

二、结果呈现

```
最大总利润: 1146.448 元

各产品各加工路径的最优产量:
产品III（A2B2）: 324 件
产品II（A1B1）: 0 件
产品II（A2B1）: 500 件
产品I（A1B1）: 0 件
产品I（A1B2）: 629 件
产品I（A1B3）: 571 件
产品I（A2B1）: 0 件
```

```
各产品的总产量:
产品 I总产量: 1430 件
产品 II总产量: 500 件
产品III总产量: 324 件
```

三、代码展示

```python
from pulp import LpProblem, LpMaximize, LpVariable, lpSum, value

prob = LpProblem("Factory_Production_Plan", LpMaximize)

x111 = LpVariable("产品 I（A1B1）", lowBound=0, cat='Integer')
x112 = LpVariable("产品 I（A1B2）", lowBound=0, cat='Integer')
x113 = LpVariable("产品 I（A1B3）", lowBound=0, cat='Integer')
x121 = LpVariable("产品 I（A2B1）", lowBound=0, cat='Integer')
x122 = LpVariable("产品 I（A2B2）", lowBound=0, cat='Integer')
```

```
x123 = LpVariable("产品 I（A2B3）", lowBound=0, cat='Integer')
x211 = LpVariable("产品 II（A1B1）", lowBound=0, cat='Integer')
x221 = LpVariable("产品 II（A2B1）", lowBound=0, cat='Integer')
x322 = LpVariable("产品 III（A2B2）", lowBound=0, cat='Integer')

prob += (
    0.375 * x111 + 0.3026 * x112 + 0.4 * x113 +
    0.4003 * x121 + 0.3279 * x122 + 0.4253 * x123 +
    0.65 * x211 + 0.8611 * x221 +
    0.6844 * x322
), "Total_Profit"

prob += (
    5 * (x111 + x112 + x113) + 10 * x211 <= 6000
), "A1_Available_Time"

prob += (
    7 * (x121 + x122 + x123) + 9 * x221 + 12 * x322 <= 10000
), "A2_Available_Time"

prob += (
    6 * (x111 + x121) + 8 * (x211 + x221) <= 4000
), "B1_Available_Time"

prob += (
    4 * (x112 + x122) + 11 * x322 <= 7000
), "B2_Available_Time"

prob += (
    7 * (x113 + x123) <= 4000
), "B3_Available_Time"

prob.solve()

print(f"最大总利润：{value(prob.objective)} 元")
print("\n 各产品各加工路径的最优产量：")
for v in prob.variables():
    print(f"{v.name}: {v.varValue:.0f} 件")

prod1_total = x111.varValue + x112.varValue + x113.varValue +
x121.varValue + x122.varValue + x123.varValue
prod2_total = x211.varValue + x221.varValue
prod3_total = x322.varValue
```

```
print("\n 各产品的总产量：" )
print(f"产品Ⅰ总产量：{prod1_total:.0f} 件")
print(f"产品Ⅱ总产量：{prod2_total:.0f} 件")
print(f"产品Ⅲ总产量：{prod3_total:.0f} 件")
```

1-4

一、求解过程

（1）决策变量

设 xij 为第 i 类货物装载到第 j 个货舱的重量（t），其中：i=1,2,3,4（对应货物 1~4），j=1,2,3（对应前舱、中舱、后舱）共 12 个决策变量。

（2）目标函数

总利润 = 各类货物的单位利润 × 装载量之和

即 Z=3100(x11+x12+x13)+3800(x21+x22+x23)+3500(x31+x32+x33)+2850(x41+x42+x43)取最大值

（3）约束条件

1. 货舱重量约束

① 前舱总重量：x11+x21+x31+x41≤10

② 中舱总重量：x12+x22+x32+x42≤16

③ 后舱总重量：x13+x23+x33+x43≤8

2. 货舱体积约束

① 前舱总体积：480x11+650x21+580x31+390x41≤6800

② 中舱总体积：480x12+650x22+580x32+390x42≤8700

③ 后舱总体积：480x13+650x23+580x33+390x43≤5300

3. 货物总量约束

① 货物 1 总重量：x11+x12+x13≤18

② 货物 2 总重量：x21+x22+x23≤15

③ 货物 3 总重量：x31+x32+x33≤23

④ 货物 4 总重量：x41+x42+x43≤12

4. 飞机平衡约束

三个货舱实际重量必须与最大重量成比例:

（x11+x21+x31+x41）/10=（x12+x22+x32+x42）/16=（x13+x23+x33+x43）/8

等价于两个等式约束:

① 16(x11+x21+x31+x41)=10(x12+x22+x32+x42)

② 8(x12+x22+x32+x42)=16(x13+x23+x33+x43)

5. 非负约束

所有决策变量 xij≥0

（4）求解模型

Python 的 pulp 库（求解线性规划问题）

二、结果呈现

```
最大总利润: 121515.79 元

各货物在各货舱的最优装载量:
货物1在前舱: 0.0000 t
货物1在中舱: 0.0000 t
货物1在后舱: 0.0000 t
货物2在前舱: 7.0000 t
货物2在中舱: 0.0000 t
货物2在后舱: 8.0000 t
货物3在前舱: 3.0000 t
货物3在中舱: 12.9474 t
```

```
货物3在后舱: 0.0000 t
货物4在前舱: 0.0000 t
货物4在中舱: 3.0526 t
货物4在后舱: 0.0000 t
```

三、代码展示

```python
from pulp import LpProblem, LpMaximize, LpVariable, lpSum, value

prob = LpProblem("Cargo_Plane_Loading", LpMaximize)
```

```
x = [[LpVariable(f"x{i+1}{j+1}", lowBound=0, cat='Continuous') for j in
range(3)] for i in range(4)]

profit = [3100, 3800, 3500, 2850]
prob += lpSum(profit[i] * lpSum(x[i][j] for j in range(3)) for i in
range(4)), "Total_Profit"

prob += lpSum(x[i][0] for i in range(4)) <= 10, "Front_Weight_Limit"
prob += lpSum(x[i][1] for i in range(4)) <= 16, "Middle_Weight_Limit"
prob += lpSum(x[i][2] for i in range(4)) <= 8, "Rear_Weight_Limit"

volume = [480, 650, 580, 390]
prob += lpSum(volume[i] * x[i][0] for i in range(4)) <= 6800,
"Front_Volume_Limit"
prob += lpSum(volume[i] * x[i][1] for i in range(4)) <= 8700,
"Middle_Volume_Limit"
prob += lpSum(volume[i] * x[i][2] for i in range(4)) <= 5300,
"Rear_Volume_Limit"

max_cargo = [18, 15, 23, 12]
for i in range(4):
    prob += lpSum(x[i][j] for j in range(3)) <= max_cargo[i],
f"Cargo{i+1}_Total_Limit"

front_weight = lpSum(x[i][0] for i in range(4))
middle_weight = lpSum(x[i][1] for i in range(4))
rear_weight = lpSum(x[i][2] for i in range(4))
prob += 16 * front_weight == 10 * middle_weight, "Balance_Front_Middle"
prob += 8 * middle_weight == 16 * rear_weight, "Balance_Middle_Rear"

prob.solve()

print(f"最大总利润: {value(prob.objective):.2f} 元")

print("\n 各货物在各货舱的最优装载量: ")
cargo_names = ["货物 1", "货物 2", "货物 3", "货物 4"]
hold_names = ["前舱", "中舱", "后舱"]
for i in range(4):
    for j in range(3):
        print(f"{cargo_names[i]}在{hold_names[j]}: {value(x[i][j]):.4f}
t")
```